Intelligent Information Systems
Security Engineering

Next-Gen Exfiltration: new techniques of packet modification in covert channels, implementation and detection

Submitted by:

Muhammad Nour Ali & Alward Munder

Supervised by:

Dr. Muhammad Alchaita - Eng. Mohammad Yamen Hallak

2026

1. Introduction

1.1 Covert Channels in Networking

A covert channel is an unintended or unauthorized intra-system channel that enables two cooperating entities to transfer information in a way that violates the system's security policy but does not exceed the entities' access authorizations. [1]

In the networking context, a covert channel typically exploits some aspect of a communication protocol to encode and convey secret information. The fundamental requirement is a shared, observable medium through which a sender modulates data that a receiver can subsequently decode, while the exchange remains indistinguishable from benign traffic to an external observer. [2]

Covert channels in networks are broadly classified into two categories:

- **Storage channels** — information is encoded in discrete, finite fields within a protocol header or data structure (e.g., the IP Identification field or an HTTP header value).
- **Timing channels** — information is encoded in the temporal relationships between events, such as inter-packet delays whose intervals represent binary states.

These channels pose a serious threat to network security because they enable data exfiltration, command-and-control (C2) communication, and covert information transfer in environments where conventional security controls would otherwise detect or block direct, overt communication. [3]

1.2 Historical Background

The formal study of covert channels originated in operating system security research and has evolved through four broadly identifiable phases, each driven by advances in both attack capability and defensive countermeasures.

Phase 1 — Early Theoretical Foundations (1970s–1980s). Following Lampson's 1973 paper, the U.S. Department of Defense Trusted Computer System Evaluation Criteria (TCSEC, 1983) [4] formally required covert channel analysis as a prerequisite for high-assurance system certification. Research in this era was predominantly theoretical, cataloguing channels into two families: storage channels (which relay bits by modifying a shared, observable storage object) and timing channels [4] (which encode information in the intervals between observable events).

Phase 2 — Network-Layer Covert Channels (1990s). The proliferation of the internet shifted research attention to the TCP/IP stack. Craig Rowland's (1997) work on IP and TCP header manipulation demonstrated that reserved, unused, or low-variance fields — such as the IP identification field, TCP sequence numbers, and TCP urgent pointer — could carry hidden data while appearing to be normal traffic to shallow packet inspection tools [5]. These techniques required no special privileges and introduced the notion of protocol-steganographic channels.

Phase 3 — Application-Layer and Protocol-Tunnelling Channels (2000s–2010s). As network perimeter defenses matured and deep-packet inspection (DPI) became commercially viable, attention moved up the protocol stack to the application layer. Researchers demonstrated covert channels hidden within DNS queries, ICMP echo payloads, and eventually HTTP headers. The HTTP channel attracted particular interest because HTTP traffic traversed virtually every enterprise firewall without inspection and because the protocol's specification left numerous header fields semantically unconstrained in their values or lengths.

Phase 4 — Detection-Aware and Adversarial Channels (2010s–Present). The deployment of statistical and behavioral Network Detection and Response (NDR) systems — most notably RITA (Real Intelligence Threat Analytics) [6] and Zeek (formerly Bro) [7] — created a new adversarial dynamic. Modern covert channel research now simultaneously advances both the design of detection-evading channels and the development of machine-learning-based detection frameworks, framing the problem as an active arms race between covert communication and network forensics.

1.3. HTTP as Covert Channel

HTTP was selected as the target protocol for this research on the basis of five properties that collectively make it both a highly capable covert channel medium and a practically significant threat surface.

Ubiquity and Firewall Permissibility. HTTP and HTTPS traffic operates on TCP ports 80 and 443, [8] which are universally permitted through enterprise firewalls and network perimeter defenses by operational necessity. Unlike ICMP-based, DNS-based, or raw-socket channels — all of which may be subject to egress filtering in hardened environments — HTTP traffic is rarely blocked or throttled, making it a reliable vehicle for covert communication even in adversarial network environments.

No Elevated Privileges Required. Unlike network-layer channels that require raw socket access (and therefore root or administrator privileges on most operating systems), HTTP connections are initiated by standard userland processes using conventional socket APIs. This drastically lowers the operational barrier: any compromised process running in a user context can establish an HTTP covert channel without triggering privilege-escalation alerts. [9]

Non-Fixed-Length Header Fields. HTTP/1.1 specifies a rich set of request and response headers, many of which are optional or permit arbitrarily variable-length values. Fields such as User-Agent, Accept-Language, Cookie, X-Forwarded-For, Cache-Control, and custom X- extension headers can accommodate encoded payloads of varying length while remaining syntactically RFC-compliant. This structural flexibility is fundamental to maximizing the information-carrying capacity — or *bandwidth* — of the covert channel.[9]

Semantic Richness Provides Natural Cover Traffic. HTTP traffic is inherently voluminous and heterogeneous: web browsers, API clients, software update services, and telemetry agents all generate continuous HTTP traffic with widely varying header compositions. A well-designed HTTP covert channel can be engineered to closely mimic the statistical and behavioral fingerprint of legitimate web traffic, making its traffic indistinguishable to a casual or automated observer.

2. Literature Review

2.1 Overview of Covert Communication Research

Recent research on covert communication has explored techniques for transmitting information through network protocols, host execution behavior, storage systems, and hardware resources. These studies demonstrate that covert channels can exploit different properties of legitimate system operations to encode information while attempting to avoid detection. However, their capacity, detectability, reliability, and implementation requirements vary considerably depending on the protocol and embedding mechanism employed.

For the purposes of this research, particular attention is given HTTP protocol and related web protocols. Other approaches are considered to establish a broader understanding of covert communication techniques and to identify common limitations that may inform the design of an HTTP-based system.

2.2 HTTP-Based Covert Channels

Bistarelli et al. (2024), in Design and Implementation of a Covert Channel Based on HTTP Headers, investigated a network storage channel that encodes information through the values of eight standard HTTP request-header fields. According to the comparison matrix, their implementation achieves a capacity of up to 608 bits per request and operates in standard user space without requiring elevated privileges. The reported evaluation also indicates successful evasion of Suricata and limited detection by RITA for payloads below 7 KB. Nevertheless, the approach exhibits important limitations: high-volume transfers can trigger behavioral beaconing alerts, web proxies may remove or modify headers, and the implementation does not provide a return communication channel. [10] These limitations are directly relevant to the present research, which investigates expanding HTTP embedding opportunities, evaluating behavioral detection, and incorporating server-to-client communication.

2.3 Covert Channels in Related Network Protocols

Huang et al. (2025), in QuicReflo: Exploiting Multi-Flow Redundancy for Restoration-Free Covert Communication over QUIC, investigate a different approach based on redundancy across concurrent HTTP/3 flows. Their technique exploits spatiotemporal relationships between flows and uses a model to identify suitable embedding opportunities. The matrix reports an average throughput of 28 bytes per packet and a machine-learning classifier accuracy of approximately 60.2% under the reported evaluation. [15] However, the method depends on intermediate-node access and may lose effectiveness as website layouts change. Although this approach demonstrates the potential of modern web protocols for covert communication, its infrastructure requirements distinguish it from an HTTP/1.1 implementation designed to operate through ordinary user-space applications.

Velinov et al. (2026), in Steganography in the QUIC Communication Protocol, present a framework categorizing 20 potential covert channels across QUIC protocol features. The matrix identifies an implemented proof of concept involving the Retry Token field and reports approximately 44.6 bits per second under the evaluated configuration. It also indicates that most proposed channels remain theoretical and that certain scenarios require access to TLS session keys. This study illustrates the importance of distinguishing theoretical embedding opportunities from experimentally validated implementations. [16] Similarly, the present research must evaluate its proposed HTTP header and request-body techniques experimentally rather than assuming that additional fields automatically produce higher effective throughput or lower detectability.

Gorrochategui et al. (2026), in STUN Comprehension-Optional Attributes as a Covert Channel: Design, Implementation, and Detection, demonstrate another application-layer approach using optional STUN attributes to carry encapsulated Ethernet frames. The matrix reports successful evasion of several security products and substantial throughput on embedded devices. However, the approach also presents detectable characteristics, including high-frequency polling, unusual attribute sizes, and entropy-related anomalies. [17] Although STUN differs from HTTP, this work highlights an important general principle: bypassing individual security controls does not necessarily make a covert channel undetectable. Behavioral and statistical characteristics must also be evaluated.

2.4 Alternative Covert-Channel Mechanisms

Other studies demonstrate that covert communication is not restricted to application-layer protocols. Filho et al. (2025), in Exploiting Temporal Variability in TCP Timestamps for Covert Channel Design, investigate encoding information through TCP timestamp behavior. Their work considers both a timestamp-based approach and a more reliable transmission model involving acknowledgments and buffering. The comparison matrix reports a capacity of approximately 0.11–0.16 bits per packet for the timestamp-based model and a delivery success rate of 98–100% for the reliable model under the reported lossy conditions. [18] However, packet loss, buffering overhead, and middlebox normalization remain limitations. These findings emphasize the trade-off between covert-channel capacity, reliability, and robustness.

Wendzel et al. (2024–2025), in DYST (Did You See That?): An Amplified Covert Channel That Points To Previously Seen Data, propose a technique that uses small signaling packets to reference previously observed legitimate background traffic. Rather than transmitting all the information directly, the method exploits traffic already shared by the communicating parties. Its limitations include dependence on shared background traffic and sensitivity to packet reordering and loss. [20] This work illustrates an alternative strategy for reducing the amount of explicitly encoded information, although it does not directly establish the effectiveness of HTTP header embedding.

Additional studies extend covert-channel research to host and hardware environments. Alhelal and Al-Khatib (2024) investigate covert transmission through Java exception-handling behavior and report high throughput, while identifying exposure to host-based monitoring and application-specific dependencies.[20] Jiang and Wang (2024), in Sync+Sync: A Covert Channel Built on fsync with Storage, examine timing differences caused by contention in persistent storage operations. [21] Bostancı et al. (2025), in LeakyHammer, investigate timing channels associated with RowHammer defense mechanisms in DDR5 memory. [22] Natkaniec and Kępowicz (2025) explore a multi-mechanism covert channel using IEEE 802.11e Wi-Fi MAC behavior, [23] while Chen et al. (2025), in TCCM: Unobservable Blockchain-Based Covert Communication, investigate embedding information into Bitcoin transaction fields. [24] Collectively, these studies demonstrate the diversity of covert-channel mechanisms, but their host, hardware, wireless, storage, and blockchain-specific assumptions differ from the application-layer focus of the present research.

2.5 Comparative Analysis and Research Gaps

Title, Authors & Year	Target Protocol & Layer	Channel Type	Core Summary	Research Gap
Design and Implementation of a Covert Channel based on HTTP Headers **Bistarelli et al. (2024)**	HTTP / L7	Storage (PS11)	Modulates 8 HTTP GET request headers (up to 608 bits/req)	RITA beaconing alerts on large files; GET-only; W3C proxy header stripping.
QuicReflo: Exploiting Multi-Flow Redundancy for Restoration-Free Covert Communication over QUIC **Huang et al. (2025)**	QUIC / HTTP/3 (L4–L7)	Multi-Flow Spatio-Temporal	Hierarchical diffusion models (backbone + site-adaptive) precompute Redundancy Space Vectors across concurrent flows for restoration-free insertion (~28 B/pkt).	Site RSVs suffer model drift on site layout updates; requires multi-flow sessions & GPU pre-training.
Steganography in the QUIC Communication Protocol **Velinov et al. (2026)** JUCS, Vol. 32, No. 2	QUIC / HTTP/3 (L4–L7)	Taxonomy & Retry Token Storage	Categorizes 20 novel QUIC covert channels; implements PoC modulating Retry Token field in Retry packets with AES (44.6 bps avg, up to 102.4 kbps on NGINX).	19 of 20 channels remain theoretical without empirical code; remote C2 (AS2) requires sharing QUIC session keys or endpoint access.
STUN comprehension-optional attributes as a covert channel: Design, implementation, and detection **Gorrochategui et al. (2026)** Computers & Security, Vol. 170	STUN / UDP (L7)	L2 Tunnel	Encapsulates L2 Ethernet frames into STUN optional TLV attributes (0x8FAD) ignored per RFC 5389; adaptive polling for full-duplex C2 (19.6–41.9 Mbps).	High payload entropy (6.25 bits/B vs 4.39 max benign) flagged by Zeek/Suricata 8 entropy rules; high polling rate (2050 req/min).
Creating and Developing a High-Throughput Covert Channel via Program Execution **Alhelal & Al-Khatib (2024)** IEEE Access, Vol. 12	JVM / L7	Storage	Exploits Java Array Exception via Huffman + Hex coding (~23-52 Mbps)	High host/SIEM visibility; vulnerable to thread noise; requires vulnerable Java code.
Detection of an Imperceptible HTTP-based Covert Channel **Menachi et al. (2024/2025)**	HTTP / L7	Storage	Kernel Netfilter hooks hiding data in Cache-Control & Content-Encoding	Lacks empirical source code; unresolved TCP sequence realignment; no speed metrics

Title, Authors & Year	Target Protocol & Layer	Channel Type	Core Summary	Research Gap
Exploiting Temporal Variability in TCP Timestamps for Covert Channel Design **Filho et al. (2025)** IEEE Access, Vol. 13	TCP Timestamps / L4	Storage / Timing	Modulates TCP Timestamp LSB parity (statistically near-0 RMSD)	Low bit-rate; packet loss corrupts frame; cleared if middlebox rewrites timestamps.
DYST (Did You See That?): An Amplified Covert Channel That Points To Previously Seen Data. **By: Steffen Wendzel, Tobias Schmidbauer, Sebastian Zillien, Jörg Keller (2024-2025)**	Network / Transport / Cross-Layer	Points to unaltered, legitimate third-party background traffic previously observed by sender and receiver using tiny 1-bit signaling packets (e.g. ARP/DNS).	Introduces 'history covert channels' and 'covert channel amplification' (CAF). Instead of sending modified payload data, the sender transmits tiny 1-bit signaling packets that point to matching hashes in legitimate background traffic. Achieves high stealthiness as the data channel is 100% legitimate traffic.	1. Highly dependent on shared observable background traffic (broadcast LANs, unencrypted multicast, or shared router paths). 2. Sensitive to packet reordering/dropouts, requiring heavy 'robust mode' delays that reduce throughput. 3. Multi-pointer throughput optimization (>32-64 pointers) increases signal density, making the channel detectable via compressibility and KS tests.
Sync+Sync: A Covert Channel Built on fsync with Storage **By: Qisheng Jiang, Chundong Wang (ShanghaiTech University) (2024)**	Storage / File System Layer	Exploits contention on shared file system structures (e.g. Ext4 journal) and disk I/O dispatch queues caused by concurrent fsync() system calls.	Reveals a new timing covert channel at the persistent storage layer using fsync(). Reaches 20,000 bps capacity with ~0.40% error rate across files, containers, VMs, and disks without sharing any data or file access permissions.	1. Heavy concurrent I/O workloads calling fsync (e.g., Varmail, RocksDB) introduce substantial noise, elevating error rates. 2. File system features like Ext4 'Fast Commit' or kernel I/O throttling reduce contention and degrade channel performance. 3. Higher symbol durations required for noise-resilience drastically reduce raw transmission speed.
StegoEDCA: An Efficient Covert Channel for Smart Grids Based on IEEE 802.11e Standard **By: Marek Natkaniec, Paweł Kępowicz (AGH University of Krakow) (2025)**	Data Link Layer (Wi-Fi MAC 802.11e)	Combines 4 MAC mechanisms: EDCA Access Categories (AC), Duration/ID field shifting, TXOP frame sequence encoding, and EDCA backoff parity.	Proposes a multi-mechanism MAC covert channel for Smart Grid Wi-Fi communications. Simultaneously encodes data across QoS queues, frame duration shifts, TXOP bursts, and backoff slots, achieving covert throughput exceeding 50 kb/s under heavy load.	1. Strongly dependent on frame length: long frames reduce covert bitrate significantly. 2. Frame aggregation (A-MPDU) in newer Wi-Fi standards (802.11n/ac/ax) alters TXOP frame counts, requiring specific threshold recalibration. 3. High network congestion with many competing stations lowers covert throughput due to backoff collision interference.

Title, Authors & Year	Target Protocol & Layer	Channel Type	Core Summary	Research Gap
TCCM: Unobservable Blockchain-Based Covert Communication **By: Zhuo Chen, Liehuang Zhu, Peng Jiang, Zijian Zhang, Jialing He (2025)**	Application / Blockchain Layer	Embeds secret data into Bitcoin mainnet transaction fields using reversible steganography while bypassing transaction data mining inspection.	Establishes an unobservable, high-capacity covert channel over permissionless blockchains (Bitcoin mainnet). Transmits MB-level files within ~3 minutes at ~$1.47 per 100 MB while resisting statistical steganalysis and deep learning detectors.	1. Financial cost depends on Bitcoin network transaction gas/miner fees. 2. High network congestion on the blockchain can delay transaction confirmation and covert data retrieval. 3. On-chain transaction pruning or modified node transaction validation rules could disrupt retrieval.
LeakyHammer: Understanding and Mitigating Covert Channel and Side Channel Vulnerabilities Introduced by RowHammer Defenses **By: F. Nisa Bostancı, Oğuzhan Canpolat, Ataberk Olgun, İsmail Emir Yüksel, Konstantinos Kanellopoulos, Mohammad Sadrosadati, A. Giray Yağlıkçı, Onur Mutlu (ETH Zurich) (2025)**	Hardware / Memory Subsystem Layer	Exploits timing latencies introduced by hardware RowHammer defenses (DDR5 PRAC - Per Row Activation Counting, and RFM - Refresh Management) to create side/covert channels.	Uncovers that hardware RowHammer defenses in DDR5 introduce timing covert channels (LeakyHammer) with up to 48.7 Kbps channel capacity. Demonstrates cross-process secret leakage and website fingerprinting via DRAM activation backoffs.	1. Application-induced memory contention causes row buffer conflicts, slightly increasing error rates (~0.03 error rate under high noise). 2. Mitigation schemes like FR-RFM (Flush-on-Refresh) eliminate the channel but introduce performance overhead in highly vulnerable DRAMs. 3. Requires precise microarchitectural timing measurement capabilities.

3. Project Goals

This research pursues three interrelated objectives, spanning the full arc from covert channel construction to detection.

Goal 1 — Extend the Bandwidth of the HTTP Covert Channel. Prior work has explored only a limited subset of HTTP header fields for covert data embedding, focusing almost exclusively on GET request headers and a narrow range of well-known fields. This project systematically investigates novel embedding opportunities across a broader field space — including POST request bodies and headers, response headers, and previously unstudied optional or extension fields — to increase the aggregate information-carrying capacity of the channel. Dual encoding schemes (hexadecimal and Base64) are applied across multiple headers simultaneously to maximize per-request throughput.

Goal 2 — Reduce Detectability. Existing HTTP covert channel implementations exhibit statistical anomalies that are reliably detected by behavioral NDR systems such as RITA. This project targets detectability reduction through two complementary mechanisms: (a) *timing jitter* — the introduction of randomized inter-request delays (50–300 ms) to disrupt the regular periodicity that beaconing detection algorithms exploit; and (b) *method balancing* — dynamic variation of the GET/POST request ratio to prevent statistically atypical method distributions that trigger anomaly flags. Together, these mechanisms aim to bring the channel's traffic profile within the statistical envelope of legitimate web browsing behavior.

Goal 3 — Develop an Intelligent Detection System. While Goals 1 and 2 advance the analytical capability of the channel for research purposes, the ultimate objective of this project is defensive. The final deliverable is an intelligent detection system capable of identifying HTTP covert channel traffic despite active evasion measures. This system is designed around passive network monitoring via the Zeek Network Security Monitor — leveraging http.log, conn.log, and header field distributions — combined with behavioral analytics from RITA NDR, which evaluates beaconing regularity scores and GET/POST ratios. The resulting system provides a principled, empirically grounded baseline for HTTP covert channel detection in enterprise environments.

4. Research Gaps

A survey of the existing literature reveals three principal gaps that this research is specifically designed to address.

Gap 1 — Exclusive Focus on GET Requests. The overwhelming majority of prior HTTP covert channel studies restrict their scope to GET requests. While GET is the most common HTTP method, this focus leaves a significant portion of the protocol's attack surface unexplored. POST requests — which permit a structured request body in addition to headers — offer additional embedding opportunities in both header fields and body content. Furthermore, the presence of an exclusively GET-heavy traffic pattern is itself a detectable anomaly in environments where POST traffic normally constitutes a substantial fraction of web activity. A channel that deliberately and dynamically balances GET and POST methods is simultaneously more bandwidth-capable and less statistically distinctive. Extending covert channel research to encompass the full HTTP method space is therefore both a technical contribution and a practical evasion necessity.

Gap 2 — High Anomaly Scores in RITA. Several prior HTTP covert channel implementations have been evaluated against RITA (Real Intelligence Threat Analytics), a widely-deployed open-source behavioral NDR framework that scores traffic on the basis of beaconing regularity, connection frequency, and volumetric patterns. These implementations have consistently produced high RITA beaconing scores, indicating that the channels remain

detectable in practice by deployed defenses. No existing published work has specifically targeted the *reduction* of RITA anomaly scores as a primary design objective, nor has any work proposed principled timing and method variation as a countermeasure. This gap represents both a practical limitation of the current state of the art and an opportunity for a novel and measurable research contribution.

Gap 3 — Unidirectionality of the Channel. Existing HTTP covert channel research has focused almost exclusively on the client-to-server direction: transmitting secret data from a compromised endpoint to a C2 receiver. The return path — from the C2 server back to the compromised client — has received little systematic attention. A unidirectional channel is functionally limited: it cannot support interactive C2 operations, acknowledgment-based reliability, or bidirectional data exchange. This project addresses the bidirectionality gap by incorporating a decoy HTTP response mechanism — wherein the server returns an HTTP 200 OK response carrying dummy HTML or JSON content that may itself embed covert return-path data — effectively completing a full-duplex covert communication loop over standard HTTP.

5. Proposed Solution and System Architecture

The proposed system implements a bidirectional HTTP covert channel with integrated passive detection monitoring. It comprises three functionally distinct subsystems, illustrated at the macro level in the architecture diagram (Figure 1) and in operational detail in the prototype flowchart (Figure 2).

5.1 High-Level Architecture

At the macro level, the system models a realistic enterprise adversarial scenario. A **Compromised Workstation** (the secret sender) communicates with a **Desired Server / C2 Backend** (the secret receiver) using RFC-compliant HTTP/1.1 GET and POST requests whose headers carry covert data. All traffic transits through a layer of **Enterprise Perimeter Defenses** — specifically, a Zeek NSM and RITA Behavioral NDR monitoring stack operating in passive out-of-band mode. The C2 backend returns a **Decoy Response** to the workstation via the same path, completing the bidirectional loop while maintaining the appearance of ordinary web browsing activity.

Critically, the monitoring layer is passive: it analyses a copy of the traffic via a network mirror port rather than sitting inline. This models the most common real-world deployment of such detection systems and ensures that the detection pipeline does not introduce latency or interfere with the channel's operation.

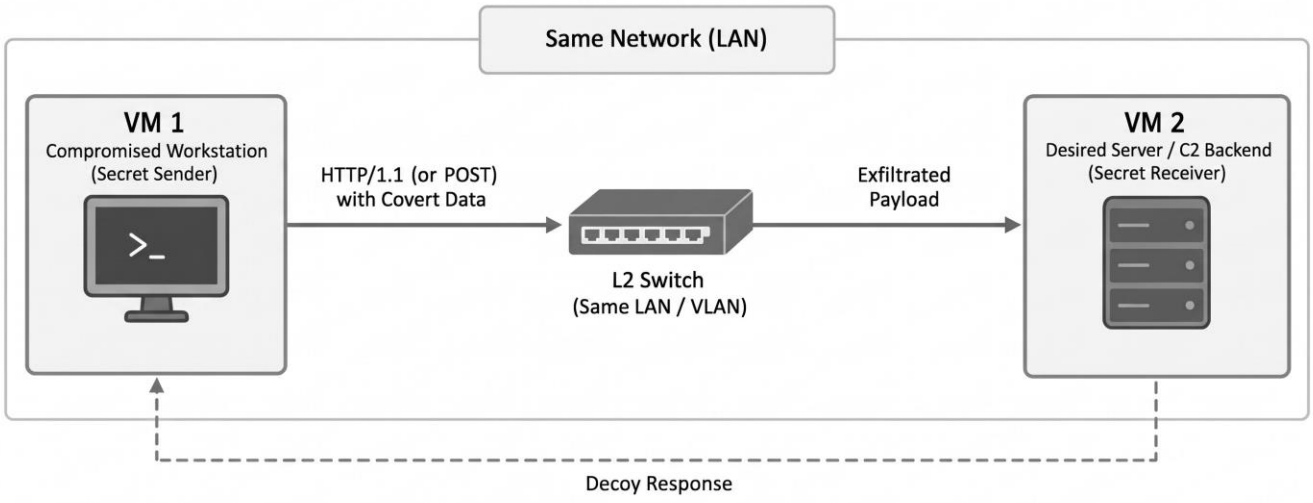


Figure 1: project architecture diagram

5.2 Prototype Flowchart (Figure 2)

The detailed prototype flowchart decomposes the system into three interacting subsystems:

Subsystem A — Compromised Client (Python Sender)

The sender side operates in four sequential steps:

1. **Read Secret Payload File** — The sender reads a target file (binary or text) that constitutes the secret data to be exfiltrated.
2. **Slice Payload and Encode into Headers** — The payload is segmented into fixed-size chunks; each chunk is encoded (hexadecimal or Base64) and inserted into one or more HTTP header fields selected from the available embedding space.
3. **Apply Timing Jitter and Method Balancing** — A randomized delay (50–300 ms) is introduced between successive requests to disrupt beaconing signatures. The HTTP method (GET or POST) is dynamically selected according to a configurable ratio to prevent anomalous method-distribution patterns.
4. **Send HTTP GET/POST Request** — The fully constructed request, carrying the encoded covert payload within its headers, is transmitted to the target server.

Subsystem B — Target Web Server and C2 Receiver (Python Receiver)

The receiver side operates in four corresponding steps:

5. **Web Server Ingress / Dummy Application** — An HTTP server receives the incoming request and serves dummy web pages, maintaining the external appearance of a legitimate web service.
6. **Python Receiver / Header Extractor** — A parallel receiver process inspects incoming requests, extracts the covert header fields, and passes the encoded chunks to the reconstruction pipeline.
7. **Reconstruct Secret Payload** — The extracted chunks are decoded (hex/Base64) and reassembled in order to recover the original secret file.
8. **Return HTTP Decoy Response** — The server returns an HTTP 200 OK response containing dummy HTML or JSON content. This response constitutes the return channel of the bidirectional link and may itself carry covert return-path data in its response headers.

Subsystem C — Passive Network Monitoring (Out-of-Band)

Running independently and non-intrusively, the detection subsystem operates across three layers:

- **Network Mirror Port** — Raw line-rate traffic is captured passively from a mirrored network port without inserting any inline component.
- **Zeek NSM Engine** — The captured traffic is processed by Zeek, which parses and logs HTTP sessions (http.log), connection metadata (conn.log), and individual header field values — producing the structured telemetry that the detection analytics consume.
- **RITA NDR Analytics** — RITA ingests Zeek's logs and evaluates the traffic against a suite of behavioral indicators, including beaconing regularity scores and GET/POST method ratios. The output of this layer determines whether the channel's evasion mechanisms — timing jitter and method balancing — are sufficient to suppress anomaly scores below detection thresholds.



Figure 2: prototype flowchart

5.3 Research Contribution of the Architecture

The proposed architecture makes a direct and measurable contribution to each of the three project goals. The dual-method (GET/POST), multi-field embedding strategy addresses Goal 1 by expanding the usable embedding space beyond prior GET-only approaches. The timing jitter and method-balancing mechanisms address Goal 2 by specifically targeting the RITA beaconing and method-ratio detection signals identified in Gap 2. Finally, the integrated Zeek/RITA monitoring subsystem provides the empirical detection baseline required to evaluate evasion efficacy and, ultimately, to train and validate the intelligent detection system described in Goal 3.

Citations

[1] NIST, “Covert channel” (Glossary entry), National Institute of Standards and Technology, Computer Security Resource Center, n.d.

[2] Rowland: C. H. Rowland, “Covert Channels in the TCP/IP Protocol Suite,” First Monday, 2(5), Psionic Software, Inc., 1997. https://firstmonday.org/ojs/index.php/fm/article/view/528

[3] Orange Book : U.S. Department of Defense / National Computer Security Center, “Trusted Computer System Evaluation Criteria” (Orange Book, CSC-STD-001-83), DoD Computer Security Center, 1983. https://irp.fas.org/nsa/rainbow/std001.htm

[4] U.S. Department of Defense / National Computer Security Center. (1983). Trusted Computer System Evaluation Criteria (CSC-STD-001-83). DoD Computer Security Center.

[5] Rowland, C.H. (1997). Covert channels in the TCP/IP protocol suite. First Monday, 2(5). https://doi.org/10.5210/fm.v2i5.528

[6] Active Countermeasures. (2016). RITA: Real Intelligence Threat Analytics (open-source framework). GitHub. https://github.com/activecm/rita

[7] Paxson, V. (1999). Bro: A system for detecting network intruders in real-time. Computer Networks, 31(23–24), 2435–2463. https://doi.org/10.1016/S1389-1286(99)00112-7

[8] Fielding, R. & Reschke, J. (Eds.). (2014). Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing (RFC 7230). Internet Engineering Task Force. https://doi.org/10.17487/RFC7230

[9] Lubacz, J., Mazurczyk, W. & Szczypiorski, K. (2014). Principles and overview of network steganography. IEEE Communications Magazine, 52(5), 225–229. https://doi.org/10.1109/MCOM.2014.6815904

[10] IANA, “Service Name and Transport Protocol Port Number Registry,” Internet Assigned Numbers Authority, n.d.

[11] R. Fielding, M. Nottingham, and J. Reschke (Eds.), “HTTP Semantics” (RFC 9110), Internet Engineering Task Force, 2022.

[12] “HTTP Header Reordering-based Covert Channel Protocol,” 2023

[13] Huang et al. QuicReflo: Exploiting Multi-Flow Redundancy for Restoration-Free Covert Communication Over QUIC, IEEE,2026

[14] Velinov et al, Steganography in the QUIC Communication Protocol, Journal of Universal Computer Science, 2026

[15] Gorrochateguin et al. STUN Comprehension-Optional Attributes as a Covert Channel: Design, Implementation, and Detection, ScienceDirect, 2026

[16] Filho et al, Exploiting Temporal Variability in TCP Timestamps for Covert Channel Design, ResearchGate, 2025

[17] Wendzel et al. DYST (Did You See That?): An Amplified Covert Channel That Points To Previously Seen Data, IEEE, 2024

[18] Alhelal et al. Creating and Developing a High-Throughput Covert Channel via Program Execution, IEEE, 2024

[19] Jiang et al. Sync+Sync: A Covert Channel Built on fsync with Storage, USINEX, 2024

[20] Bostancı et al. Understanding and Mitigating Covert Channel and Side Channel Vulnerabilities Introduced by RowHammer Defenses, arxiv, 2025

[21] Marek Natkaniec and Paweł Kępowicz, StegoEDCA: An Efficient Covert Channel for Smart Grids Based on IEEE 802.11e Standard, MDPI, 2025

[22] Chen et al. Exploring Unobservable Blockchain-Based Covert Channel for Censorship-Resistant Systemsm ResearchGate, 2025